

Data Preparation

Aufbauend auf der im vorherigen Kapitel durchgeführten explorativen Analyse und der Aufarbeitung von Qualitätsmängeln werden in diesem Kapitel die Schritte zur systematischen Aufbereitung der Datenbasis beschrieben. Ziel dieser Phase ist es, eine fehlerfreie, konsistente und für den KI-Agenten optimal nutzbare Grundlage zu schaffen. Der Prozess folgt den etablierten Phasen des CRISP-DM-Modells: Datenkonsolidierung, Data Cleansing und Feature Engineering (siehe Grundlagenkapitel CRISP-DM).

Datenkonsolidierung

Zunächst habe ich die fünf GitHub-Repositories, die die Datenquellen dieser Arbeit darstellen, in eine kontrollierte und einheitliche Umgebung überführt. Dies geschah, indem für jede Anwendung ein privater Fork auf einem dedizierten GitHub-Account angelegt wurde. Dieser Schritt war strategisch erforderlich, um zum einen die **Kontrolle** über die für die Bereinigung notwendigen Änderungen zu erhalten, zum anderen die **Sicherheit** sensibler Anmeldeinformationen durch GitHub Actions Secrets zu gewährleisten und schließlich den programmatischen **Zugriff** für den Agenten mittels eines dedizierten Tokens zu ermöglichen.

Data Cleansing

Anschließend wurden die im Data Understanding identifizierten Mängel in der Datenqualität behoben, um eine durchgängige Funktions- und Deploy-Fähigkeit der Anwendungs-Repositories sicherzustellen. Dies umfasste die manuelle Erstellung der fehlenden *mta.yaml*-Dateien, die für ein Deployment auf die Cloud Foundry Runtime essenziell sind, sowie die Korrektur veralteter oder fehlerhafter Build-Skripte in den *package.json*-Dateien.

Eine automatisierte Generierung der *mta.yaml* durch den Agenten wurde bewusst ausgeschlossen. In dieser Konfigurationsdatei werden anwendungsspezifische Architekturentscheidungen, wie die Ressourcenallokation, getroffen, die maßgeblichen Einfluss auf Performance und Kosten haben. Solche kritischen Design-Entscheidungen sollen von einem Entwickler verifiziert werden, weshalb der Agent den Prozess lediglich unterstützen, nicht aber vollständig autonom durchführen soll.

Feature Engineering

Der letzte und für das Modeling zentrale Schritt war das Feature Engineering. Dabei geht es darum, aus den umfangreichen Rohdaten der Repositories jene Merkmale (Features) auszuwählen, die für die nachgelagerte Modellerzeugung relevant sind. Im Gegensatz zu klassischem Feature Engineering, das häufig Transformationen numerischer oder kategorischer Werte umfasst, besteht der Fokus hier in der gezielten Reduktion der Repository-Daten auf inhaltlich relevante Artefakte, die das LLM benötigt, um eine valide CI/CD-Pipeline generieren zu können.

Basierend auf den Erkenntnissen aus dem Data Understanding wurde definiert, dass zur Generierung einer CI/CD-Pipeline gezielt drei zentrale Merkmale (Features) aus jedem Repository extrahiert werden müssen:

1. Die Projektstruktur in Form der Ordner- und Dateihierarchie.
2. Der Inhalt der Konfigurationsdatei *package.json*.
3. Der Inhalt des Deployment-Deskriptors *mta.yaml*.

```

RELEVANT_FILES = ["mta.yaml", "package.json"]

def get_relevant_files(self):
    """Lädt relevante Dateien aus dem Repository."""
    collected = {}
    url = f"{self.base_url}/repos/{self.owner}/{self.repo}/contents"
    items = self._request("GET", url)

    for item in items:
        if item["name"] in RELEVANT_FILES and item.get("download_url"):
            file_resp = requests.get(item["download_url"])
            if file_resp.status_code == 200:
                collected[item["name"]] = file_resp.text
    return collected

```

Quelltext 1: Python-Funktion zur Extraktion relevanter Dateien via GitHub API.

Die Extraktion erfolgt durch gezielte Leseoperationen (siehe Quelltext 1) über die GitHub API, autorisiert durch den zuvor konfigurierten Token. Das Ergebnis dieses Prozesses ist ein Objekt, welches alle relevanten Dateinamen und deren Inhalte bereithält.

Nach Abschluss der Datenvorbereitung liegt somit eine bereinigte und konsistente Datenbasis vor. Die fünf CAP-Anwendungen sind nun funktionsfähig und ein definierter Satz an Merkmalen wurde als Input für den Agenten extrahiert. Diese aufbereitete Grundlage ist die unmittelbare Voraussetzung für die im folgenden Kapitel beschriebene Phase des **Modelings**, in der der KI-Agent entwickelt wird, um diesen Input zu verarbeiten und die CI/CD-Pipeline zu generieren.